Orma: measuring discretion in on-ledger private credit

Alexandre Lemiere Andrea Gonzalez

13 September 2026, corrected 13 October 2026, extended 25 October 2026

Abstract

This is the technical paper for Orma. It states the arithmetic the protocol runs on, so that a reader can check it rather than take it on trust, and without reading JavaScript. Every formula is implemented in the repository, and every measured figure was read from live XRP Ledger Devnet state during the De Vinci Blockchain XRPL Lending Protocol Hackathon, Paris, 12 and 13 September 2026, against `rippled` 3.4.0-rc5. The central claim is combinatorial: total first-loss capital consumed across a fixed set of defaults depends on the order in which they are declared, that order is chosen unilaterally by the party whose capital is consumed, and a closed form makes the size of the choice computable from public state. On the measured facility it is worth 0.20 XRP out of 0.70.

Section 7, added after the XRPL hackathon in New York, 24 and 25 October 2026, states the arithmetic of Orma Desk, which lends against vault shares at their withdrawal value. Its figures were read on Devnet during that event, not in Paris.

1 The setting

Sections 1 to 6 are the Paris paper: the information layer, built and measured in September 2026. Section 7 is the New York extension. It puts that layer to one use, a lender's: Orma Desk lends XRP against shares of a closed-ended vault, values the pledge only at the held unit value of Section 2, prices the default of each late loan of the vault before it happens, and countersigns a loan only after reading the ledger again. The pledge sits in escrow toward a two-of-three account, so no single party can move it. The rules it keeps are stated as seven invariants, each with the check that tests it.

1.1 The objects

A *Vault* (XLS-65) holds pooled assets and issues a share token. To carry lending it must be closed-ended: `VaultKind` set to 1, with a `SubscriptionDate` and a `RedemptionDate`. Without those three fields `LoanBrokerSet` returns `tecNO_PERMISSION`, and `VaultKind` is immutable, so this is decided at creation.

The vault runs through Subscription, Investment and Redemption, and `VaultWithdraw` returns `tecTOO_SOON` for the whole of the Investment phase regardless of available liquidity: inability to withdraw then is a protocol fact, not a distress signal, and nothing below treats it as one.

A *LoanBroker* (XLS-66) attaches to exactly one vault. It draws capital from the vault, originates *Loan* objects, and posts *first-loss capital*, called cover, which is liquidated ahead of the vault's own assets when a loan is written off. The broker declares the credit events: impairment, un-impairment, default. The share token is an `MPTokenIssuance` whose `OutstandingAmount` is the unit count and whose metadata the issuer may write, which is what lets a share point at its own valuation (Section 6).

1.2 The identity that generates the problem

Two protocol facts combine into one asymmetry. First, only the `LoanBroker` owner may impair, un-impair or default a loan. Second, the `LoanBroker` owner is necessarily the vault owner: no other account can create the broker, and XLS-75 permission delegation returns `temMALFORMED` across the entire lending suite, so the authority cannot be split off to a third party even voluntarily. That second fact was re-probed on live Devnet during the event, against a positive control on the same two accounts in the same run; Section 1.4 gives the six results and the control.

Therefore the party who decides *when* a loss is recognised, the party who decides *in what order* a set of losses is realised, and the party whose own first-loss capital is consumed when they are, are one account. Write τ for the timing choice and σ for the ordering choice. Section 2 measures what τ is worth, Section 4 what σ is worth.

Both move value from unit holders to the manager, and neither is visible to a consumer using the standard tooling. Orma measures both from public state, publishes the result as a native XLS-47 Oracle object and as a pointer inside the share token, and lets an independent third party gate subscriptions on it through XLS-70 credentials. The enforcement layer contains no arithmetic and is not analysed here.

1.3 Notation

symbol	meaning	ledger field
A	total assets held by the vault	`Vault.AssetsTotal`
A_v	assets not currently lent out	`Vault.AssetsAvailable`
L	loss recognised but not yet deducted	`Vault.LossUnrealized`
S	units outstanding	`MPTokenIssuance.OutstandingAmount`
D	broker's total drawn debt	`LoanBroker.DebtTotal`
C	first-loss capital posted	`LoanBroker.CoverAvailable`
$c_{\min}$	minimum cover rate	`LoanBroker.CoverRateMinimum`
c_{liq}	liquidation cover rate	`LoanBroker.CoverRateLiquidation`
p_i	principal of exposure i	`Loan.PrincipalOutstanding`

Units. Monetary quantities are integer drops, 1 XRP = 10^6 drops, carried on the wire as decimal strings of up to 19 significant digits. All arithmetic runs at 40 significant digits with `decimal.js`; no monetary value passes through `Number()` or `parseFloat`, because a 53-bit mantissa cannot represent every value the protocol admits. $c_{\min}$ and c_{liq} are in units of 10^{-5}, so a stated `10000` means 0.10: not 10000, and not 100.

Accounting basis. All facilities here run `LEVersion` = 1, cash basis. The consequence that matters is that origination moves `DebtTotal`, not `AssetsTotal`: lending does not reduce the assets the vault reports holding. This is load-bearing for Proposition 2.

1.4 The measured facilities

Three live Devnet facilities are used below. Their ids are collected here so the text can name them in words and every figure quoted later can be re-derived from public state.

object	id
Meridian Trade Finance I, vault	`864C5A2DDCED78C6198B0703169D533B747A56E5F9398D832033F79E37E8C835`
Meridian, impairing transaction	`075FE6D2E0F29919AF477A2A8F581A680805A006138BB967D8434611E49229C3`
Kestrel Bridge Financing II, vault	`24EAA01AD4CE70D8ABB4ACB4255DC4C216B8AF3B9E7315DB8052BFC1E5810089`
Kestrel, loan broker	`8DB104980C31CBE9F196508E7D748E8C31FB783CD4E0A3620AE0B476CBDEA3FA`
Kestrel, first default	`FB23EC5313F85FA157D56966E0B0B1665068905988D925C70BB0E7C35C3B221B`
Kestrel, second default	`DE49DF0D6136B96AD28B253CD32100E5715078FD2D0AE6D1DB747AFECB803D1F`
Calder Structured Credit III, broker	`1C160EFFCA8208DFD8CE9B065A3C3C41CCA80758FBC3099BE27AACFBD1478173`
Calder, share token issuance	`000000014D667775372D5B78E07FFF294678C7F9CE82AFBC`
Kestrel, broker and vault owner	`rD26gw5goDqU784caePMoA82givcJBjStk`
delegation probe, granting account	`rG1rKEDnW2HWkQdjjREtC78kPFKpwGu2H2`
delegation probe, delegate	`rHK8rwcvywYrv6AJ8HW1J8byKBiJEBRYH4`

The delegation probe. The claim in Section 1.2 that lending authority cannot be delegated was re-proven on Devnet during the event, at 07:29 UTC on 13 September 2026, from the granting account to the delegate listed above. The first row below is a positive control: a `DelegateSet` granting `Payment`, submitted from the same account to the same delegate in the same run, which the network accepted.

delegated permission	result	id
`Payment` (control)	`tesSUCCESS`	`547DA71992D469ACA345A0B3CEAD07864E2801919201B256EA606C3A7910AF17`
`LoanManage`	`temMALFORMED`	never reached a ledger
`VaultCreate`	`temMALFORMED`	never reached a ledger
`VaultDeposit`	`temMALFORMED`	never reached a ledger
`LoanBrokerSet`	`temMALFORMED`	never reached a ledger
`LoanSet`	`temMALFORMED`	never reached a ledger
`VaultWithdraw`	`temMALFORMED`	never reached a ledger

The six absent hashes are not a hole in the evidence, they are the only shape the evidence can take. A `tem` result is a rejection at submission: the transaction is refused before consensus, is never included in a ledger and is never assigned a slot, so it has no hash to cite and no ledger entry to look up.

A hash for `LoanManage` delegation would mean the grant had been accepted. What a reader checks instead is the control, and the control is what rules out the alternative explanation that the account, the delegate, the build or the probe itself was broken. Delegation works on this network. The lending suite is specifically excluded from it.

2 The two readings of net asset value

2.1 Definitions

Definition 1. The *naive* unit value, which is what a tool reading the headline asset figure computes, and the *held* unit value, which is what a holder can redeem against, are

$$\mathrm{NAV}_{\mathrm{naive}} = \frac{A}{S}, \tag{1}$$

$$\mathrm{NAV}_{\mathrm{held}} = \frac{A - L}{S}, \tag{2}$$

both defined as 0 when $S = 0$.

They agree exactly until a loss is recognised and diverge the instant one is. Divergence is reported in basis points,

$$\delta \;=\; \mathrm{round}\!\left(10^4 \cdot \frac{\mathrm{NAV}_{\mathrm{naive}} - \mathrm{NAV}_{\mathrm{held}}}{\mathrm{NAV}_{\mathrm{naive}}}\right) \tag{3}$$

$$\;=\; \mathrm{round}\!\left(10^4 \cdot \frac{L}{A}\right), \tag{4}$$

to the nearest integer basis point, halves away from zero. The second equality holds for $S \neq 0$, $A \neq 0$, and shows that divergence does not depend on the unit count at all: it is the recognised loss as a fraction of the reported book.

Remark. $\mathrm{NAV}_{\mathrm{naive}}$ can *rise* while a vault is emptied, because whenever $L > 0$ a withdrawal burns units faster than it removes assets.

2.2 Why the naive reading is not merely lazy

The obvious objection is that any competent indexer nets the recognised loss, so the naive reading is a straw man. It is not, and the reason is a property of the ledger's metadata rather than of anyone's code.

Proposition 2 (The empty change set)**.** *Let V be a Vault under* `LEVersion` $= 1$ *whose* `LossUnrealized` *is* 0 *just before a* `LoanManage` *transaction with the impair flag set, and let that transaction succeed and set* `LossUnrealized` *to some $L' > 0$. Then the* `PreviousFields` *object of V's node in that transaction's metadata is empty.*

Proof. The metadata for a modified node lists in `PreviousFields` the previous value of each field that was present on the object before the transaction and changed. An optional (`SoeDefault`) field at its type's default, 0 for the amount-valued fields of a Vault, is not stored on the object, so a field the transaction creates has no previous value to record.[1]

Take each field in turn. `AssetsTotal` is unchanged: under cash-basis accounting origination recorded the principal in `DebtTotal`, and recognising a loss on an outstanding loan writes no asset movement. `AssetsAvailable` is unchanged: nothing is lent, repaid, deposited or withdrawn. The share issuance is untouched, so S is unchanged, and every remaining field of the Vault is static configuration. The single field whose value differs is `LossUnrealized`, moving from 0 to L'; but at 0 it was not stored on the object, so it has no previous value and is not emitted. No other field differs, hence `PreviousFields` is empty. ■

Corollary 3. *Let f be any function of the* `PreviousFields` *object alone. Then f takes the same value on the metadata of the impairing transaction as on the metadata of a transaction that changed nothing about the vault. An indexer maintaining vault state by applying metadata diffs therefore continues to report* $\mathrm{NAV}_{\mathrm{naive}} = \mathrm{NAV}_{\mathrm{held}}$ *indefinitely after the first impairment, and reports no event at all.*

The scope is narrow and still damaging. It applies to the *first* impairment of a previously healthy vault; on the second, L is already non-zero, the field is emitted normally, and a diffing consumer catches up. The blind spot is exactly the transition from healthy to impaired, which is the one a lender cares about most.

The remedy costs one round trip: Orma never diffs `PreviousFields` but polls each vault object every 4 seconds, re-reads it outright and recomputes both readings from the current values. `FinalFields` in the same metadata also carries the correct post-state, so the failure is specific to the diff.

2.3 Measured

Facility *Meridian Trade Finance I*, impairing transaction as listed in Section 1.4, result `tesSUCCESS`.

field	healthy, before	final, after
`AssetsTotal`	51,000,000	51,000,000
`AssetsAvailable`	41,000,000	41,000,000
`LossUnrealized`	0	10,000,000
`PreviousFields` emitted for the Vault node: `{}`		

[1] Corrected 13 October 2026: the rule is presence, not value (`xrpl-dev-portal#3926`).

With $S = 51{,}000{,}000$ units,

$$\mathrm{NAV}_{\mathrm{naive}} = \frac{51{,}000{,}000}{51{,}000{,}000} = 1.000000, \tag{5}$$

$$\mathrm{NAV}_{\mathrm{held}} = \frac{41{,}000{,}000}{51{,}000{,}000} = 0.803922, \tag{6}$$

the second rounded to nearest at the 6 decimal places the valuation is reported to. The divergence is $10^4 \cdot 10{,}000{,}000/51{,}000{,}000 = 1960.7843\ldots$, so $\delta = 1961$ bp: the reported unit value overstates recoverable value by 19.61% of the reported figure.

The base is worth stating, because the instinctive one is the other: δ is defined as a fraction of $\mathrm{NAV}_{\mathrm{naive}}$, which is the right base for a reader who starts from what their tool displayed, while the same gap as a fraction of recoverable value is $10{,}000{,}000/41{,}000{,}000 = 24.39\%$. Neither is wrong and the two are not interchangeable, so every percentage below names the base it is taken against.

Two readers pointed at the same transaction on the same ledger at the same instant return 1.000000 and 0.803922.

3 Cover liquidated on a default

When the broker declares a default on a loan of principal p, the ledger liquidates

$$T(p) \;=\; \min\Big(\big\lceil D \cdot c_{\min} \cdot c_{\mathrm{liq}} \big\rceil,\; p,\; C\Big) \tag{7}$$

of posted cover. Three properties of (7) matter and none is obvious from the prose specification.

The double product. Both rates are 10^{-5} units, so both are divided by 10^5. At $c_{\min} = c_{\mathrm{liq}} = 10000$ the coefficient is not 0.10 but $0.10 \times 0.10 = 0.01$. Reading one of the two as a plain multiplier overstates liquidatable cover tenfold.

The ceiling. The product is rounded up to whole drops, not truncated. At the scales used here the argument is an exact integer and the ceiling is inert, but it is implemented because the ledger implements it and it becomes visible at odd rates.

The base. The base is the broker's *total* outstanding book D, not the principal p of the exposure that defaulted: a default is charged against the size of the whole book at the instant it is declared. That choice is what makes Section 4 non-trivial, and it also makes concentration and under-collateralisation one defect seen from two sides, since a book of one very large loan liquidates no more cover on its default than a book of many small ones liquidates on the first of them.

Worked, from live state. The *Calder Structured Credit III* broker, with $D = 10{,}000{,}000$ drops, $C = 5{,}000{,}000$ drops and $c_{\min} = c_{\mathrm{liq}} = 10000$:

$$c = 0.10 \times 0.10 = 0.01, \tag{8}$$

$$\lceil 10{,}000{,}000 \times 0.01 \rceil = 100{,}000 \text{ drops} = 0.10 \text{ XRP}. \tag{9}$$

The broker posted 5 XRP of cover, of which 0.10 XRP is reachable by any single default at the current book size: the stranded fraction $\max(0, C - \lceil D c_{\min} c_{\mathrm{liq}} \rceil)/C$ is 0.98. The separately defined cover *requirement* $\lceil D c_{\min} \rceil = 1{,}000{,}000$ drops is met with room to spare, so a consumer checking the requirement alone concludes the facility is well covered.

4 The ordering result

4.1 Statement

Fix a broker and write $c = c_{\min} \cdot c_{\text{liq}}$. Let D_0 be the book at the first default, $p_1, \dots, p_k$ the principals of k exposures that will all be defaulted, and σ a permutation of $\{1, \dots, k\}$ giving the order in which the broker declares them. `DebtTotal` is decremented by p as each default settles, so the book facing step i is $D_{i-1} = D_0 - \sum_{j<i} p_{\sigma(j)}$.

Write $T(\sigma, k)$ for total cover consumed across the k declarations, under three regularity assumptions that hold on the measured facility and that Section 8 revisits.

(A1) $c\,D_{i-1}$ is an integer number of drops at every step, so the ceiling in (7) is inert.

(A2) Cover never runs out: $C \geq T(\sigma, k)$ for every σ considered.

(A3) Each principal is at least the step demand, $p_{\sigma(i)} \geq c\,D_{i-1}$, so the per-step min never binds on p.

Lemma 4 (Closed form)**.** *Under (A1) to (A3),*

$$T(\sigma, k) \;=\; c\left[\, kD_0 \;-\; \sum_{i=1}^{k} (k-i)\, p_{\sigma(i)} \right]. \tag{10}$$

Proof. Under the assumptions the amount liquidated at step i is exactly $c\,D_{i-1}$, so

$$T(\sigma, k) = \sum_{i=1}^{k} c\,D_{i-1} = c \sum_{i=1}^{k} \Big(D_0 - \sum_{j<i} p_{\sigma(j)} \Big) \tag{11}$$

$$= c\left[kD_0 - \sum_{i=1}^{k} \sum_{j<i} p_{\sigma(j)} \right]. \tag{12}$$

Exchange the order of summation. The term $p_{\sigma(j)}$ appears once for each i with $j < i \leq k$, that is $k - j$ times, so $\sum_{i=1}^{k} \sum_{j<i} p_{\sigma(j)} = \sum_{j=1}^{k} (k-j)\, p_{\sigma(j)}$, which is the claim after renaming the index. ∎

Only the second term depends on σ, and it enters with a minus sign, so minimising total cover consumed is the same problem as maximising $\Sigma(\sigma) = \sum_i (k-i)\, p_{\sigma(i)}$: a sum of the principals against the fixed, strictly decreasing weight vector $(k-1, k-2, \dots, 1, 0)$.

Theorem 5 (Ordering)**.** *Under (A1) to (A3), $T(\sigma, k)$ is minimised when σ declares the exposures in non-increasing order of principal, largest first, and maximised when it declares them in non-decreasing order, smallest first. The extremal values are*

$$T_{\min} = c\left[kD_0 - \sum_{i=1}^{k} (k-i)\, p^{\downarrow}_{(i)} \right], \tag{13}$$

$$T_{\max} = c\left[kD_0 - \sum_{i=1}^{k} (k-i)\, p^{\uparrow}_{(i)} \right], \tag{14}$$

where $p^{\downarrow}$ and $p^{\uparrow}$ are the principals sorted decreasing and increasing.

Proof. By Lemma 4 it suffices to show Σ is maximised by the decreasing arrangement and minimised by the increasing one. This is the rearrangement inequality; the exchange argument is given in full because it also yields uniqueness up to ties.

Suppose σ is not non-increasing in principal. Then there are adjacent positions i and $i+1$ with $a = p_{\sigma(i)} < p_{\sigma(i+1)} = b$. Let σ' be σ with those two transposed. Every other position contributes identically, so

$$\Sigma(\sigma') - \Sigma(\sigma) = \big[(k-i)\,b + (k-i-1)\,a\big] - \big[(k-i)\,a + (k-i-1)\,b\big] \tag{15}$$

$$= b - a \;>\; 0, \tag{16}$$

because the two weights differ by exactly 1. Every adjacent transposition that repairs an inversion therefore strictly increases Σ and strictly decreases T. A permutation of k elements has finitely many inversions and each such transposition removes one, so the process terminates, and it can terminate only at an arrangement with no inversions, that is the non-increasing one.

Hence the decreasing order attains the maximum of Σ and no other order class does. The minimising case is identical with the inequality reversed. ■

Corollary 6 (The spread)**.**

$$T_{\max} - T_{\min} \;=\; c \sum_{i=1}^{k} (k-i)\big(p^{\downarrow}_{(i)} - p^{\uparrow}_{(i)}\big). \tag{17}$$

For $k = 2$ this collapses to $c\,(p_{\max} - p_{\min})$.

Corollary 7 (Stated plainly)**.** *The party who chooses σ is the* `LoanBroker` *owner, who is necessarily the vault owner. The capital consumed by $T(\sigma, k)$ is that same party's posted cover, and every drop of cover not consumed is a drop of loss falling instead on the vault's unit holders. The party choosing the order is exactly the party whose capital the choice spares, and the amount spared, against the investor-optimal order, is $T_{\max} - T(\sigma, k)$.*

4.2 Measured on Devnet

Facility *Kestrel Bridge Financing II*, ids in Section 1.4. Two loans of 30 XRP and 10 XRP, $D_0 = 40$ XRP, $C = 10$ XRP posted, $c_{\min} = c_{\text{liq}} = 10000$ so $c = 0.01$. The broker declared the large loan first. Both orders, in XRP:

order	step	book D_{i-1}	principal $p_{\sigma(i)}$	$\lceil cD_{i-1} \rceil$	cumulative
largest first	1	40	30	0.40	0.40
(observed)	2	10	10	0.10	**0.50**
smallest first	1	40	10	0.40	0.40
(counterfactual)	2	30	30	0.30	**0.70**

The closed form agrees. With $k = 2$, $D_0 = 40$, $c = 0.01$ and weights $(1, 0)$,

$$T(\text{largest first}) = 0.01\,[\,80 - 30\,] = 0.50, \tag{18}$$

$$T(\text{smallest first}) = 0.01\,[\,80 - 10\,] = 0.70, \tag{19}$$

and Corollary 6 gives the spread directly as $0.01 \times (30 - 10) = 0.20$ XRP without computing either endpoint.

The ledger agrees too: reconstructed from public transaction history alone, across the two default transactions of Section 1.4 `CoverAvailable` moved 10,000,000 → 9,600,000 → 9,500,000 drops, consuming 400,000 and 100,000 drops, total 500,000 drops = 0.50 XRP. Three independent computations agree: the step-by-step simulation, the closed form, and the ledger itself.

Identical losses were realised in both rows; the only difference is the order of two transactions eight seconds apart. The 0.20 XRP was borne by the vault's investors, and the party who chose the order is the party it spared. Measured against the 0.50 XRP the first-loss capital did absorb, the investor-optimal order would have absorbed 40% more; measured against that 0.70 XRP, the choice withheld 28.6% of it.

4.3 Sequence score

Observed consumption is placed between the two extremes:

$$\phi \;=\; \frac{T_{\text{observed}} - T_{\min}}{T_{\max} - T_{\min}} \;\in\; [0, 1], \tag{20}$$

defined as 1 when $T_{\max} = T_{\min}$, which happens when all principals are equal or fewer than two defaults have occurred.

$\phi = 0$ is the investor-worst sequence and $\phi = 1$ the investor-best. For Kestrel, $\phi = (0.50 - 0.50)/(0.70 - 0.50) = 0.0000$. A separate ordinal conduct score starts at 100 and penalises -35 for $\phi < 0.25$, -15 for $0.25 \le \phi < 0.75$, -20 per loss written off with no prior impairment and -10 per withdrawal of cover; Kestrel scores 25, grade E.

Reconstructing this history needs a join that is easy to miss. A `LoanManage` transaction carries no `LoanBrokerID`, and an impairment does not touch the `LoanBroker` object at all, so filtering `account_tx` on the broker object silently drops every impairment and returns a history in which no loss was ever signalled before it was taken. The `Loan` node in the same metadata carries `FinalFields.LoanBrokerID`, which closes the join at no extra request. Before that fix, a rule written to reward early disclosure was penalising it.

5 The scoring model

5.1 Five factors, no weights

Five quantities are measured from ledger state, each mapped independently onto a 20-step ordinal ladder. No weighted average is taken at any stage: a weighted average lets a strong factor pay for a broken one, which is precisely the failure mode a credit assessment must not have.

Liquidity. A_v/A.

First-loss adequacy. Liquidatable cover against the exposure it must absorb,

$$\text{adequacy} \;=\; \frac{\min\left(\lceil D c_{\min} c_{\text{liq}} \rceil,\; C\right)}{\max\left(L,\; P_{\text{distressed}}\right)}, \tag{21}$$

defined as 1 when the denominator is zero. The denominator takes the *larger* of the recognised loss and the principal of exposures that are overdue, impaired or defaultable. Dividing by L alone would grade a vault concealing a large overdue exposure as perfectly covered. Concealment must never improve a score; the max is what enforces that.

Concentration. $\max_i(p_i)/D$, banded on 1 minus that value so higher is better.

Recognition lag. Seconds an exposure has been past due with nothing declared. This is the direct measurement of the timing lever τ.

Redemption cliff. Projected shortfall of claims over liquidity at redemption,

$$\begin{aligned}\text{shortfall} \;=\; \max\Big(0,\; (A - L) \\ - \left(A_v + R_{\text{perf}} + \mathbf{1}[P_{\text{distressed}} > 0] \cdot C_{\text{rec}}\right)\Big),\end{aligned} \tag{22}$$

with R_{perf} the total contractual repayment of performing loans and C_{rec} the recoverable cover $\min(\lceil D c_{\min} c_{\text{liq}} \rceil, C)$. `rippled` refuses a `LoanSet` whose term overruns the vault's `RedemptionDate`, so every performing loan is contractually repaid before redemption and a projected shortfall is exactly the non-performing book net of recoverable cover. Being fully lent is therefore not a distress signal here. The bands are cut on that shortfall as a fraction of claims, shortfall$/(A - L)$, not on its size in drops.

The bands are stated in full so they can be disputed. Each threshold is a floor: the first one the value meets or exceeds gives the grade, and a value below all of them grades `D`. Recognition lag is banded on seconds rather than a ratio: 0 is AAA, under 60 is A, under 300 is BBB, under 900 is BB, beyond that CCC.

factor	quantity banded	thresholds and grades
Liquidity	A_v/A	0.95 AAA, 0.85 A, 0.70 BBB, 0.50 BB, 0.25 B, 0.05 CCC
First-loss adequacy	adequacy	1 AA, 0.50 A-, 0.20 BBB-, 0.05 BB-, 0.01 B-
Concentration	$1 - \max_i(p_i)/D$	0.80 AA, 0.60 A-, 0.40 BBB-, 0.20 BB-
Redemption cliff	$1 -$ shortfall%$/100$	1 AAA, 0.98 AA, 0.90 A, 0.75 BBB, 0.50 BB, 0.25 B, 0.10 CCC

Five grades, five statements. These five are a *disclosure* layer. Each is published on its own, with its own measured value, its own band and its own sentence of explanation, and each is meant to be read on its own. That is exactly what the refusal to average protects: because the five are never collapsed into a single number, no strong factor can pay for a weak one, and a reader who cares about only one of them does not have to unpick a blend to find it.

Section 5.2 does not combine them either. The composite there is a second and separate computation, taken over raw ledger state rather than over these grades, and the two layers answer different questions. The five grades say what the facility looks like. The composite says whether to lend against it.

5.2 Composite by ordinal notching

Let $\mathcal{L} = [\text{AAA}, \text{AA+}, \ldots, \text{D}]$ with $|\mathcal{L}| = 20$, ordered best to worst, and let $\text{idx}(g)$ be the zero-based position of grade g. Define

$$\text{notch}(g, \Delta) = \mathcal{L}\big[\ \text{clamp}(\text{idx}(g) - \Delta,\ 0,\ 19)\ \big], \tag{23}$$

$$\text{numeric}(g) = \text{round}\Big(100\big(1 - \text{idx}(g)/19\big)\Big), \tag{24}$$

so a negative Δ moves the grade down the ladder and numeric projects it to a sortable number. $\mathcal{L}$ is printed here in full, because a trace whose ladder has to be reverse engineered from worked examples is not reproducible in the sense claimed for it.

i	$\mathcal{L}[i]$	i	$\mathcal{L}[i]$	i	$\mathcal{L}[i]$	i	$\mathcal{L}[i]$
0	AAA	5	A	10	BB+	15	B-
1	AA+	6	A-	11	BB	16	CCC
2	AA	7	BBB+	12	BB-	17	CC
3	AA-	8	BBB	13	B+	18	C
4	A+	9	BBB-	14	B	19	D

A second layer, read off raw state. The composite is not a function of the five grades in Section 5.1. It is computed from the same ledger state those grades were banded from, but it reads that state raw: the notch conditions test `recogLagSeconds`, `concentration`, `coverAdequacy` and ρ as numbers against their own thresholds, never the grades those numbers were banded into.

Two of the thresholds do coincide with a band edge from Section 5.1, the recognition-lag edge at 300 seconds and the adequacy edge at 0.05, and that is deliberate rather than accidental; it is still not the same operation, because a band edge produces a published grade and a notch condition produces a step. The concentration test is not a band edge at all: the concentration bands are cut on $1 - \max_i(p_i)/D$ at 0.80, 0.60, 0.40 and 0.20, and the notch fires at $\max_i(p_i)/D \geq 0.75$, which is none of them.

Exactly one rule consults a grade, the liquidity band in the Redemption phase, which fires when liquidity has graded B or worse and the vault is in the one phase where a withdrawal ought to be succeeding. The anchor is also a grade: the composite starts at the redemption cliff grade, because for a fixed-term facility the headline question is whether claims can be met at redemption, and notches from there.

Realised destruction, named before it is used. One input to the table below is measured nowhere in Section 5.1: ρ, realised capital destruction, the share of subscribed capital that has already been written off. It belongs to this layer and not to that one because it answers a decision question, has capital already been destroyed, rather than a diagnostic one, what does the facility look like now. Section 5.3 defines it and defends its denominator. It is the heaviest term in the table, worth up to fourteen notches on its own.

The realised-destruction rows are mutually exclusive; the rest accumulate. Every notch is emitted with its rule, and against the ladder above a published trace can be replayed by hand, so a grade is reproducible rather than taken on trust.

realised destruction ρ	Δ	other conditions	Δ
$\rho \geq 75\%$	-14	recognition lag > 300 s	-2
$\rho \geq 50\%$	-12	recognition lag > 0 s and ≤ 300 s	-1
$\rho \geq 25\%$	-8	largest exposure $\geq 75\%$ of the book	-1
$\rho \geq 10\%$	-5	liquidatable cover $< 5\%$ of the exposure	-1
$\rho \geq 2\%$	-3	liquidity band B or worse, in Redemption	-2
$\rho > 0\%$	-1		

5.3 Realised capital destruction, and the denominator

Each of the five factors measures current *exposure*, and a realised loss leaves none behind: the write-off removes the asset, the provision is released, and the book reads clean. Without a memory term a facility that defaulted on four fifths of its loans scores identically to one that never lost anything, the instant the write-off settles. That was observed on Devnet, not hypothesised. The term is

$$\rho \;=\; \max\left(0,\; \frac{S - A}{S}\right), \tag{25}$$

a fraction of capital subscribed rather than of assets still held, measured against A and explicitly *not* against $A - L$.

The quantity subtracted is A, and that choice carries the whole substance of the rule. An unrealised loss L is a provision against an asset the vault still holds and may still recover: it has not destroyed capital, it has disclosed that capital may be destroyed. Counting L in ρ would notch a manager down at the moment they disclose, punishing the exact behaviour the recognition-lag factor exists to reward and rewarding a manager who says nothing.

Only a write-off, which removes the asset from `AssetsTotal` outright, is permanent destruction, and only that is counted. Par is taken as 1.0, so S is capital subscribed measured in asset units; Section 8 states when that holds.

5.4 Two worked facilities

Calder Structured Credit III, composite AA. Live state: $A = 51{,}000{,}000$, $A_v = 41{,}000{,}000$, $L = 10{,}000{,}000$, $S = 51{,}000{,}000$, $D = 10{,}000{,}000$, $C = 5{,}000{,}000$, one impaired loan of principal 10,000,000. Claims at redemption are $A - L = 41{,}000{,}000$ and liquidity at redemption is $41{,}000{,}000 + 0 + 100{,}000 = 41{,}100{,}000$, so the shortfall is 0 and the anchor is AAA.

Then $\rho = 0$ because $A \geq S$, no notch. Recognition lag is 0 because the exposure is already declared impaired, no notch. Concentration is $10^7/10^7 = 1.00 \geq 0.75$, one notch. Adequacy is $\min(10^5, 5 \cdot 10^6)/\max(10^7, 10^7) = 0.0100 < 0.05$, one notch. So AAA $\to$ AA+ $\to$ AA, and $\text{numeric}(\text{AA}) = \text{round}(100(1 - 2/19)) = 89$.

Kestrel Bridge Financing II, composite B. The same broker as Section 4.2, after both defaults settled: $A = 10{,}500{,}000$ drops against $S = 50{,}000{,}000$ units subscribed. The shortfall is 0, so the anchor is again AAA, and on the five factors alone the facility grades AAA: there is no exposure left to measure, because it was destroyed rather than provisioned.

$$\rho = \frac{50{,}000{,}000 - 10{,}500{,}000}{50{,}000{,}000} = 0.79 = 79\% \geq 75\% \implies \Delta = -14. \tag{26}$$

That 79% is a fraction of capital subscribed, not of the 10.5 XRP still held. $\text{idx}(\text{AAA}) = 0$, so the composite is $\mathcal{L}[14] = \text{B}$ with $\text{numeric}(\text{B}) = \text{round}(100(1 - 14/19)) = 26$. Fourteen notches on the strength of one term. Without it, the most damaged facility in the book would sort first.

6 Valuing a pledge

A holder may pledge units to a second lender, who then holds a token and nothing else. For u units,

$$V_{\text{held}} = u \cdot \text{NAV}_{\text{held}}, \tag{27}$$

$$V_{\text{reported}} = u \cdot \text{NAV}_{\text{naive}}, \tag{28}$$

$$\Omega = V_{\text{reported}} - V_{\text{held}}, \tag{29}$$

with Ω the overstatement the second lender carries if they price the collateral off the reported figure. By the identity in Section 2, $\Omega = uL/S = (\delta/10^4) \cdot V_{\text{reported}}$ up to the rounding of δ.

The lender reaches NAV_{held} from the token alone in five steps, none requiring trust in whoever computed it: read the `MPTokenIssuance`, decode its metadata, check conformance, find the valuation pointer the issuer declared, follow it. The instrument carries the address of its own valuation basis.

6.1 The haircut applies to the held value

A lender's policy discount $h \in [0, 1)$ applies to the held value, $V_{\text{lendable}} = V_{\text{held}}(1 - h)$. This is not a matter of taste. A haircut is calibrated to the volatility of a correctly measured value: it answers how far a true value might fall before the lender can liquidate. Applying it to V_{reported} instead gives

$$V_{\text{reported}}(1 - h) = \big(V_{\text{held}} + \Omega\big)(1 - h) \tag{30}$$

$$= V_{\text{lendable}} + \Omega(1 - h), \tag{31}$$

so a residual $\Omega(1 - h)$ of pure misstatement survives into the lender's book, scaled down but never removed. Equivalently, the haircut that merely undoes the misstatement, lending nothing against volatility at all, is $h^* = 1 - V_{\text{held}}/V_{\text{reported}} = \delta/10^4$, again a fraction of the reported figure. A haircut absorbs volatility. It does not absorb a misstatement.

Against $\text{NAV}_{\text{held}} = 0.803922$ and $\text{NAV}_{\text{naive}} = 1.000000$, a pledge of $u = 1{,}000{,}000$ units:

quantity	XRP
V_{reported}	1.000000
V_{held}	0.803922
overstatement Ω	**0.196078**
V_{lendable} at $h = 0.20$, correctly applied	0.643138
haircut misapplied to V_{reported}	0.800000
residual misstatement $\Omega(1 - h)$	0.156862
break-even haircut h^*, on V_{reported}	19.61%

A conventional 20% haircut on the reported figure is, on this instrument, almost exactly consumed by the misstatement alone. It leaves 0.39 percentage points of protection against everything else that could go wrong.

7 Orma Desk: lending against shares at their withdrawal value

This section was added after the XRPL hackathon in New York, 24 and 25 October 2026. It states what Orma Desk computes and the rules it keeps. Every figure was read on XRP Ledger Devnet during that event. The lender, Harbor Credit Desk, the borrower, Tidewater Payments, and the Harbor vaults are fictional; the transactions are real.

A Devnet lending vault must be closed-ended (Section 1.1), so a holder who needs XRP before the redemption date cannot redeem and must borrow against the shares. The lender then has to price a share that no one can redeem, and Section 2 gives it two prices. Orma Desk lends against the held one only. Each figure the desk serves carries its value, a formula named from a closed table, and every input the formula used, each with the ledger object, field and id it was read from and the index and hash of the validated ledger it was read at. Amounts are decimal strings throughout, computed at 40 significant digits as in Section 1.3. The browser replays each formula on the inputs it was sent; it does not read the ledger.

7.1 The ticket

A ticket, `orma.ticket/1`, is everything one operation on a share can cost, read at one validated ledger Λ. For a pledge of u share units, write k for the vault's `Scale`, $\bar{S} = S/10^k$ and $\bar{u} = u/10^k$ for whole shares, and

$$\pi(x) = \mathrm{round}_6\big(x/\bar{S}\big), \qquad \pi(x) = 0 \text{ when } \bar{S} = 0, \tag{32}$$

$$\mathrm{val}(p) = \mathrm{round}_0\big(\bar{u}\,p\big), \tag{33}$$

both rounded to nearest, halves up: π is the unit value of Section 2 at its 6 published decimals, and val is the value of the pledge in whole drops.

Held value, naive value, and N1.

$$V_{\mathrm{held}} = \mathrm{val}\big(\pi(A - L)\big), \tag{34}$$

$$V_{\mathrm{naive}} = \mathrm{val}\big(\pi(A)\big), \tag{35}$$

$$N_1 = V_{\mathrm{naive}} - V_{\mathrm{held}}. \tag{36}$$

N_1 is what a lender overpays by valuing the shares at the value everyone reads. It is the overstatement Ω of Section 6, taken on the two rounded values.

N2, the default of each late loan alone. For each loan i of the vault that is past its due date plus its grace period and not already in default, write e_i for its exposure, `PrincipalOutstanding` under `LEVersion` $= 1$ and `TotalValueOutstanding` $-$ `ManagementFeeOutstanding` otherwise, and $\iota_i = 1$ when the loan is impaired, 0 when it is not. The cover its default would consume is (7) with e_i in place of p,

$$T_i = \min\Big(\big\lceil D \cdot c_{\min} \cdot c_{\mathrm{liq}}\big\rceil,\ e_i,\ C\Big), \tag{37}$$

and the held unit value if loan i defaults alone, now, is

$$\nu_i = \pi\Big(A - (e_i - T_i) - (L - \iota_i\, e_i)\Big) : \tag{38}$$

the default removes the exposure less the cover from the assets, and the loss of an impaired loan, already counted in L, leaves L. The loss of the pledge is

$$N_{2,i} = \begin{cases} 0 & \text{if } \iota_i = 1, \\ V_{\mathrm{held}} - \mathrm{val}(\nu_i) & \text{otherwise,} \end{cases} \tag{39}$$

since an impaired loan's loss is already in V_{held}. The ticket also states T_i and ν_i. Each entry is one default against the current state, never several at once: a joint scenario depends on the order of the defaults (Section 4).

N3, the lock-up. The seconds until the vault lets a holder redeem, `RedemptionDate` − `close_time`(Λ); the cash the vault holds today, A_v; and what the shares claim at redemption, V_{held}.

The loan. For a principal P in drops and a mandate limit m in basis points,

$$P_{\max} = \lfloor m\, V_{\text{held}}/10^4 \rfloor, \tag{40}$$

$$\text{LTV}_{\text{held}} = \lfloor 10^4 P/V_{\text{held}} \rfloor, \tag{41}$$

$$\text{LTV}_{1:1} = \lfloor 10^4 P/\bar{u} \rfloor, \tag{42}$$

the last being the same loan seen by a lender who prices a whole share at par, one drop. A pledge worth 0 at the held value has no LTV_{held}, and the ticket refuses to compute one.

Identity and staleness. The ticket's id is the SHA-256 of the canonical JSON (keys sorted, no spaces) of the share, the operation, u, the mandate's hash and the ledger index and hash, and for a loan in the test dollar also the loan's asset and the hash of the dollar terms. A second digest covers every input value its figures read except the ledger's close time and the XRP/USD median; the desk compares it at approval to tell a changed input from a later ledger.

7.2 The mandate and the refusal

The lender's rules are a mandate, `orma.mandate/1`: its id, the desk, the risk officer, four limits, the escrow policy and its issue date. Its hash is the SHA-256 of the canonical JSON of those keys and its schema name, and the risk officer anchored it on Devnet in a transaction memo. The mandate `desk-1` sets

limit	value
loan-to-value at the held value, m	53.44%
one loan	at most 4 XRP
loss if one vault loan defaults alone, $\max_i N_{2,i}$	at most 0.97 XRP
lock-up, `RedemptionDate` − `close_time`	at most 23 days

The verdict lists each limit whose value is strictly above its threshold: LTV_{held} against m, P against the single-loan limit, $\max_i N_{2,i}$ against the loss limit, and the lock-up against 86400 times the day limit. It is `PASS` when the list is empty and `BREACHED` otherwise. A second verdict judges the same rows with $\text{LTV}_{1:1}$ in place of LTV_{held}, so the ticket shows what a lender at par would decide.

At approval a breach is refused as `409 BREACHED`, not retryable, with each breach as its limit, value and threshold, and nothing is signed. The other refusals are `STALE` (an input changed since the request), `ESCROW_POLICY`, `SEQUENCE_PASSED` (the borrower's signed request can no longer be included), `INCONSISTENT_SNAPSHOT` (invariant I7 below), `LOCKED` (the kill switch is on) and, for a dollar loan, `PRICE_UNAVAILABLE`. Each is written to the decision log before it is returned.

7.3 The pledge

The borrower places its u units in a token escrow (XLS-85) whose destination is a collateral account. The desk accepts that account only with its master key disabled, no regular key, a signer list of exactly the borrower, the desk and the Orma agent at weight 1 each and quorum 2, a token holding of the pledged share, no other ledger object, no deposit authorisation, and exactly one pledge escrow, the one named. No one key moves the shares. It accepts the share only if the vault's asset is XRP, the vault is closed-ended and the share issuance flags are exactly `0x38` (escrow, trade, transfer), with neither lock nor clawback.

The escrow's window is bound to the loan. At approval, with Π the payment interval, G the grace period and n the number of payments of the `LoanSet`, `FinishAfter` must be at least $\texttt{close_time}(\Lambda) + \Pi + G$ plus a slack, so the escrow cannot be finished before the loan can default, and at most that sum plus the policy's margin; `CancelAfter` must be `FinishAfter` plus the policy's fixed offset, and at least $\texttt{close_time}(\Lambda) + n\Pi + G$ plus the slack, so the loan matures before the escrow can be cancelled. The escrow must carry no crypto-condition.

Once the escrow is finished the shares sit in the collateral account, and the Orma agent's published rule says when it co-signs their release. It finds the one desk `LoanSet`, validated `tesSUCCESS`, whose memo names the account's pledge escrow; none, or more than one, releases nothing. If that loan is in default, the shares go to the desk; if it was repaid without default, back to the borrower; in every other state, nowhere. The account must still conform and the recipient must be able to receive the shares. The agent then co-signs one transaction only: a `Payment` from the collateral account to the party the rule names, of all the shares it holds, with no `SendMax`, `DeliverMin`, `Paths`, partial payment or `Delegate`, a fee of at most 30 times the base fee, and no other field.

7.4 The countersignature and the decision log

The borrower signs a `LoanSet` whose memos name the ticket and the escrow. When the risk officer approves, the desk takes a new validated ledger Λ' and reads again at Λ' the escrow the memo names, the collateral account, the share, the borrower's account and every input of the ticket. It countersigns only if, at Λ', the escrow exists, belongs to the borrower, holds the ticket's share and units, backs no other desk loan and meets the window above; the account and the share conform; the recomputed ticket's input digest is the request's; its verdict is `PASS`; the borrower's sequence has not been used; and the kill switch is off. This is invariant I2.

Every outcome, approved, refused or declined, is one line of an append-only log,

$$e_n = \big(n,\ h_{n-1},\ \text{ticket id},\ \text{mandate hash},\ \text{decision},\ \text{code},\ \text{transaction hash},\ \Lambda',\ \text{hash}(\Lambda')\big), \quad (43)$$

$$h_n = \text{SHA256}\big(\text{canonical}(e_n)\big), \quad (44)$$

with h_0 sixty-four zeros, so altering an entry breaks the chain from that entry on. A log that cannot be opened refuses the approval before anything is signed. Margin calls, below, are entries of the same chain.

7.5 The change preview

Before the browser confirms any action, the role engine simulates its transaction on the open ledger, reads again at Λ every object the simulated metadata modifies or deletes, and serves `orma.change/1`: one row per changed field, with the value before and after, its money effect, and whether the metadata admits the change (`YES`) or hides it (`HIDDEN`), each with the line of `rippled` 3.4.1 that decides it. A field is hidden when it was absent before, at its type's default, and is present after: `PreviousFields` keeps only fields present on the original node, the rule of

Proposition 2. The known cases are `Vault.LossUnrealized` on a first impairment, `Vault.Data` on a manager's first statement, and `LockedAmount` on a holder's and an issuance's first escrow lock. `PreviousFields` is read only to decide that label, never for a value. After the transaction validates, the objects are read again and each row is marked `MATCH` or `MISMATCH`.

7.6 After the loan

The margin call. A margin policy, `orma.marginpolicy/1`, names the mandate it serves by hash, a trigger, the mandate's limit m, a cure window w and the cure, pledging more shares; it is hashed like the mandate and anchored. Each time the desk judges its positions, at most every 15 seconds and at a validated ledger, an open desk loan whose $\mathrm{LTV}_{\mathrm{held}}$ is above m moves from `NONE` to `ISSUED`, with a deadline $\texttt{close_time} + w$; back within the limit it is `CURED`; still above it after the deadline it is `EXPIRED`, and the desk may then act under its escrow policy. The collateral is not taken when the call is issued. For a principal P and pledged value H, the value missing is the least whole number of drops v with $\lfloor 10^4 P/(H+v) \rfloor \le m$,

$$v = \left\lfloor \frac{10^4 P}{m+1} - H \right\rfloor + 1, \tag{45}$$

and the call states the least whole number of share units whose value, rounded as val rounds it, covers v. Orma computes and the desk decides, so the call is labelled declared; each transition is read again at the same ledger, then becomes an entry of the decision log and an event of the feed.

The payment record. For each loan of the vault that is not closed, the ticket states its status, how long it is past due, $\max\big(0, \texttt{close_time}(\Lambda) - \texttt{NextPaymentDueDate}\big)$ seconds, and the payments missed: 0 if it is not late, otherwise $\min\big(\texttt{PaymentRemaining}, \lfloor \mathrm{late}/\Pi \rfloor + 1\big)$, because `NextPaymentDueDate` advances only when a payment is made. A defaulted loan has no due date, since the ledger deletes it, and so neither figure.

Loans in a test dollar. The desk also lends a Devnet token that stands in for RLUSD, against the same shares. The collateral rule does not change: shares of XRP vaults only, because an issued asset can be clawed back or frozen by its issuer. The XRP/USD price is the median of the XLS-47 Oracle documents the risk officer's terms name, each read at Λ, provided at least the stated number are fresh relative to $\texttt{close_time}(\Lambda)$ and lie within a stated spread of each other, and the ledger's own aggregate price over the same documents at Λ agrees. Otherwise there is no price and no dollar figure. A drop amount d is worth $\mathrm{usd}(d) = \lfloor d/10^6 \cdot x \cdot (1 - h_{fx}/10^4) \rfloor_6$ dollars, rounded down to a millionth, at price x and the terms' haircut h_{fx}, and the dollar loan-to-value is $\lfloor 10^4 P_{\$}/\mathrm{usd}(V_{\mathrm{held}}) \rfloor$.

Around the desk. `orma-mark` checks a share's valuation from its token id alone: it follows the valuation pointer of Section 6, refuses one copied from another share, recomputes the held unit value from the vault and cross-checks Orma's Oracle object. The feed derives credit events from two full re-reads of a vault, never from metadata. The universe prices, at one ledger, every vault Orma knows and every vault seen in the transaction stream since the process started.

7.7 The invariants

	rule
I1	A pledge is valued only at its held value, read at a validated ledger: every input of every figure carries that ledger's index and hash.
I2	The desk countersigns a `LoanSet` only after reading again at a new validated ledger the escrow its memo names, the collateral account and the share, and only if the recomputed ticket passes the mandate.
I3a	The collateral account has its master key disabled, no regular key and signers {borrower, desk, Orma agent} at weight 1, quorum 2.
I3b	The Orma agent co-signs only the release `Payment` its published rule names.
I4	Only shares of XRP vaults are collateral, with share flags exactly `0x38`.
I5	The browser's actions include no `LoanPay`, no lifting of an impairment and no `VaultDeposit`.
I6	A manager's statement in `Vault.Data` is labelled declared and enters no figure.
I7	No figure on an inconsistent snapshot: the vault and its share issuance are held to `rippled` 3.4.1's vault state checks, else `409 INCONSISTENT_SNAPSHOT`.

7.8 Worked: the demo loan

Harbor-1 held 50 XRP for 50,000,000 share units at `Scale` 0, the only scale an XRP vault takes, when its first loan, L1, was impaired for 10 XRP, so $\mathrm{NAV}_{\mathrm{naive}} = 1.000000$ and $\mathrm{NAV}_{\mathrm{held}} = 0.800000$ drop per unit. Tidewater pledges $u = 5{,}000{,}000$ units for $P = 2$ XRP $= 2{,}000{,}000$ drops, read at ledger 6,384,857. The vault's second loan, L2, was then defaulted on Devnet on 24 October 2026 at 22:11:21 UTC, ledger 6,389,290: 0.15 XRP of cover was drawn and the held unit value fell to 0.703000, the value computed for L2's default before it was declared.

quantity	before L2's default	after
$\pi(A - L)$, drop per unit	0.800000	0.703000
V_{held}, drops	4,000,000	3,515,000
$P_{\max}$ at $m = 53.44\%$, drops	2,137,600	1,878,416
$\mathrm{LTV}_{\mathrm{held}}$ for $P = 2$ XRP	50.00%	**56.89**%
verdict	`PASS`	`BREACHED`
$\mathrm{LTV}_{1:1}$, a lender at par	40.00%	40.00%
verdict at par	`PASS`	`PASS`

$\mathrm{LTV}_{\mathrm{held}} = \lfloor 10^4 \cdot 2{,}000{,}000/3{,}515{,}000 \rfloor = 5689$ basis points, above 5344, so the same request after the default is refused, while a lender at par, $\lfloor 10^4 \cdot 2{,}000{,}000/5{,}000{,}000 \rfloor = 4000$, would still approve it. Before the default, the N_2 formula priced L2's default at $4{,}000{,}000 - \mathrm{val}(0.703000) = 485{,}000$ drops for these units, within the 0.97 XRP limit. Had that loan of 2 XRP been outstanding against these units, the default would have opened a margin call: $v = \lfloor 2 \cdot 10^{10}/5345 - 3{,}515{,}000 \rfloor + 1 = 226{,}815$ drops of value missing, covered by 322,638 more units at 0.703000, which would bring $\mathrm{LTV}_{\mathrm{held}}$ back to $\lfloor 2 \cdot 10^{10}/3{,}741{,}815 \rfloor = 5344$ basis points.

8 Limitations

Devnet only. Every figure of Sections 1 to 6 was measured on the XRP Ledger Devnet, network id 2, against `rippled` 3.4.0-rc5. The lending amendments this depends on, XLS-65 for vaults and XLS-66 for the broker and loan objects, are not active on Mainnet, so there is no production facility to measure and no claim is made about one. The arithmetic is a property of the protocol rules rather than of Devnet and should carry over unchanged if the amendments activate. The hashes will not: Devnet is periodically reset.

A bounded history window. The conduct score and the sequence score ϕ are reconstructed from public transaction history, which the Devnet cluster prunes: on 13 October 2026 one server reported its oldest ledger as 5,193,077; the floor varies by server.[2] Behaviour before that window is invisible, so every conduct figure is a lower bound on observable misconduct over a bounded window, not a statement about a manager's whole record.

Par at 1.0**.** ρ compares A against S and so assumes one unit was issued per unit of asset subscribed. For a closed-ended vault this holds structurally rather than by convention: subscription closes before any lending is permitted, so no profit or loss can accrue between the first subscriber and the last, every subscriber enters at the same unit value, and S is exactly capital subscribed measured in asset units. `VaultKind` is immutable, so a facility cannot leave that regime later.

The assumption fails for an open-ended vault, where subscribers enter at different unit values and ρ would need the full subscription cash-flow history rather than S alone. No facility measured here can be one: `LoanBrokerSet` returns `tecNO_PERMISSION` against any vault that is not closed-ended, so a vault that lends at all is in the regime by construction. An open-ended vault would need that cash-flow history added before ρ meant anything on it.

The closed form is not the implementation. Lemma 4 drops the ceiling and the per-step min under (A1) to (A3). Where a step is capped by the defaulting principal or by remaining cover the closed form is no longer exact, although Theorem 5 stays directionally valid because a cap can only reduce a step. The implementation therefore always evaluates the iterative recursion, which mirrors the ledger step for step including the ceiling and both caps, and uses the closed form only as an independent check. On the measured facility the two agree exactly, which is the point of computing both.

Rounding, and the bands. δ is an integer basis point rounded to nearest, halves away from zero, so 1960.78 is published as 1961. One deviation is worth stating outright: the project's internal specification writes that step with a floor, while the implementation rounds, and every published figure follows the implementation. A floor would report 1960 where the ledger-facing code, the oracle object and the API all report 1961.

Unit values are likewise rounded to nearest at the 6 decimal places the valuation is reported to, and oracle values at the dimension's scale, 6 decimals for NAV and 4 for the other five. No figure here depends on a digit below those scales.

The thresholds and penalties in Section 5 are not derived from anything: they are a stated, ordinal, reproducible judgement, published in full so a reader who disagrees can say where and by how much. The two results that are not judgements are Proposition 2 and Theorem 5, which follow from the protocol rules.

[2] Corrected 13 October 2026: an earlier version gave the window as roughly 29 days. It is not a fixed number of days.

Orma Desk, also Devnet only. The figures of Section 7 were read on Devnet on 24 and 25 October 2026, with the amendments `SingleAssetVault`, `LendingProtocol`, `LendingProtocolV1_1`, `TokenEscrow`, `MPTokensV1` and `PriceOracle` enabled. What the desk relies on, and what it does not cover, is stated here rather than implied.

Keys. The borrower and the desk together can always move pledged shares, and in the demo one server holds both keys, so invariant I3a holds between distinct parties only. No release by the Orma agent runs in the demo. After a default the escrow must be finished before its `CancelAfter`, or anyone may cancel it and return the shares to the borrower.

The publisher and the browser. Orma's Oracle objects carry what Orma's key signs; `orma-mark` recomputes from the vault and only cross-checks them. The browser replays each formula on the inputs the API sent; it does not read the ledger again, and `orma-mark` and the auditor do.

Between the re-read and the loan. The `LoanSet` validates a few ledgers after the desk's re-read at Λ'. The pledge cannot leave meanwhile, since an `EscrowFinish` before `FinishAfter` and an `EscrowCancel` before `CancelAfter` are both refused, but the vault's value can move; the next ticket and the margin call see it.

Rounding of the desk's figures. The held unit value is computed at 40 digits and printed to 6 decimals. It has not been compared with the ledger's own arithmetic on a real `VaultWithdraw`, which the lock-up does not yet allow.

One default at a time. N_2 prices each late loan defaulting alone against the current state. Several defaults together cost what Section 4 says they cost, which depends on their order, and no ticket figure states it.

The dollar price. No one runs an XRP/USD oracle on Devnet. A relay of the desk copies Kraken, Bitstamp and Coinbase trades into three XLS-47 documents every 60 seconds, from one account. That demonstrates the mechanism; it is not three independent prices.